

[Edit](#)

Train and Deploy a Machine Learning Model in Azure ML

Challenge Overview

Understand the scenario

You are a Developer for Hexelo, an organization that needs to train and deploy an AI model.

In this Challenge Lab, you will train and deploy an AI model in Azure AI Machine Learning Studio. First, you will create a compute instance. Next, you will set up a machine learning job to train and register a model. Finally, you will deploy the model as an online endpoint.

Navigating the Challenge Lab

🔗 ► Understanding a Cloud Slice

🔗 ► Quick tips for navigating the Challenge Lab instructions.

[New to Challenge Labs? Click here to learn more.](#)

Create and start a compute instance

Hints Enabled

No Yes

Note About Evolving Technology

Azure AI technologies are constantly evolving as Microsoft continues to update, refine, and improve its tools and interfaces. As a result, you may notice slight differences between the instructions in this lab and what you see on your screen. Rest assured, we are actively working to keep all tutorials up to date with the latest features and interfaces. We appreciate your patience and understanding, and we apologize for any inconvenience this may cause.

- Sign in to the virtual machine as **Admin** using `{PASSWORD}` as the password.
- Open **Microsoft Edge**, go to `https://portal.azure.com`, sign in to the Microsoft Azure portal as `{USERNAME}` using `{PASSWORD}` as the password, and then dismiss all prompts.

Expand this hint for guidance on signing into the Microsoft Azure portal.

- On the Windows taskbar, in Search, enter **Edge**, and then select **Microsoft Edge** to open a new browser.
- In the Address bar, enter `https://portal.azure.com`, and then select **Enter**.
- In the Sign in dialog, enter `{USERNAME}`, and then select **Next**.
- In Enter password, enter `{PASSWORD}` and then select **Sign in**.
- Dismiss all prompts.

 Want to learn more? Review the documentation on [the Azure portal](#).

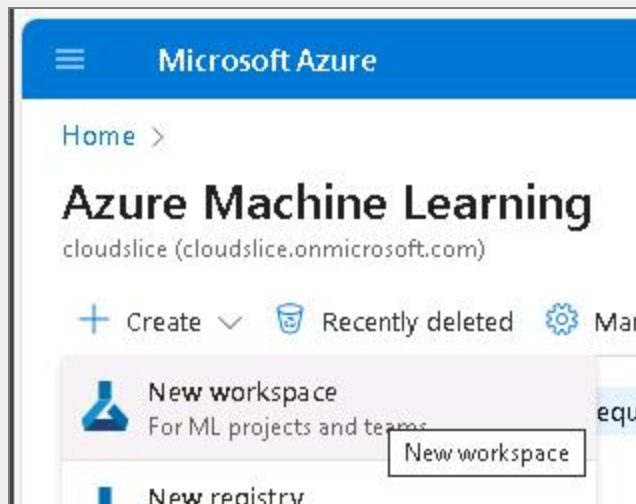
- Set up an Azure AI Machine Learning Studio workspace named `HexeloAML-workspace{LAB_INSTANCE_ID}` in the **East US** region and the **RG1lod{LAB_INSTANCE_ID}** resource group-for all other properties, use the default.

💡 Expand this hint for guidance on setting up Azure AI Machine Learning Studio. ^

- On the Microsoft Azure portal navigation bar, in *Search resources, services, and docs* (G+/, search for and select **T** Azure Machine Learning.



- On the Azure Machine Learning blade, on the command bar, select **Create | New workspace**.



- On the Azure Machine Learning blade, on the Basics page, in Resource details, in Resource group, select **{RESOURCE_GROUP_NAME}**.
- In Workspace details, in Name, enter **T** HexeloAML-workspace{LAB_INSTANCE_ID}.
- In Region, select **East US**.

Azure Machine Learning

Create a machine learning workspace

Basics

[Inbound Access](#)[Outbound Access](#)[Encryption](#)[Identity](#)[Tags](#)[Review + create](#)

Resource details

Every workspace must be assigned to an Azure subscription, which is where billing happens. You use resource groups like folders to organize and manage resources, including the workspace you're about to create.

[Learn more about Azure resource groups](#)

Subscription *

Skillable Original Content Dev 01

Resource group *

RG1lod51153434

[Create new](#)

Workspace details

Configure your basic workspace settings like its storage connection, authentication, container, and more. [Learn more](#)

Name *

HexeloAML-workspace51153434

Region *

East US

- Select **Review + create**, and then select **Create**.



Want to learn more? Review the documentation on [setting up Azure AI Machine Learning Studio](#).

- Launch Azure Machine Learning Studio from the **HexeloAML-workspace{LAB_INSTANCE_ID}** workspace.



Expand this hint for guidance on launching Azure AI Machine Learning Studio.

- On the Deployment blade, select **Go to resource**.
- On the HexeloAML-workspace{LAB_INSTANCE_ID} blade, on the Overview page, select **Launch studio**.

Work with your models in Azure Machine Learning Studio

The Azure Machine Learning Studio is a web app where you can build, train, test, and deploy ML models. Launch it now to start exploring, or [learn more about the Azure Machine Learning studio](#)

[Launch studio](#)

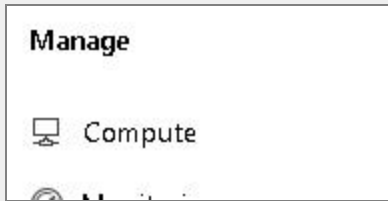


Want to learn more? Review the documentation on [launching Azure AI Machine Learning Studio](#).

- Create a **compute** instance named **hexelo-aai-mls{LAB_INSTANCE_ID}** by using **Standard_DS11_v2**.

💡 Expand this hint for guidance on creating a compute instance. ^

- On the Azure AI | Machine Learning Studio service menu, in Manage, select **Compute**.



- On the Compute page, on the Compute Instances tab, select + **New**.

Get started with Azure Machine Learning notebooks and R scripts by creating a compute instance

Choose from a selection of CPU or GPU instances preconfigured with popular tools such as VS Code, JupyterLab, Jupyter, and RStudio, ML packages, deep learning frameworks, and GPU drivers. [Learn more](#) 

+ New

- On the Create compute instance blade, in the Required settings step, in Configure required settings, in Compute name, enter T hexelo-aai-mls{LAB_INSTANCE_ID}.

Create compute instance

1 Required settings

2 Scheduling optional

3 Security optional

4 Applications optional

5 Tags optional

6 Review

Compute name * ⓘ
hexelo-aai-mls51153434

Virtual machine type ⓘ
☒ CPU ☐ GPU

Virtual machine size ⓘ
☒ Select from recommended options ☐ Select from all options

	Name ↑	Category	Workload
☒	Standard_DS11_v2 2 cores, 14GB RAM, 28GB storage	Memory optimized	Dev
☐	Standard_DS3_v2 4 cores, 14GB RAM, 28GB storage	General purpose	Clas
☐	Standard_E4ds_v4 4 cores, 32GB RAM, 150GB storage	Memory optimized	Dat
☐	Standard_D13_v2 8 cores, 56GB RAM, 400GB storage	Memory optimized	Dat

Review + Create

Back

Next

- Select **Review + Create**, and then select **Create**.

 Want to learn more? Review the documentation on [creating a compute instance](#).

 Wait until the Compute instance creation is complete.

- Confirm that the **hexelo-aai-mls{LAB_INSTANCE_ID}** compute instance is in a **Running** state-if not, **Start** the instance.

💡 Expand this hint for guidance on starting a compute instance.

- On the Compute page, in Compute Instances, confirm that **Running** is displayed in the **State** column.
- If needed, select the **hexelo-aai-mls{LAB_INSTANCE_ID}** compute instance, and then on the command bar, select **Start**.

The screenshot shows the 'Compute' page in Azure ML. At the top, there's a header 'Compute' and a message: 'The "Kubernetes clusters" tab is now where you can access previous versions of "inference compute targets using those types. Learn more about Kubernetes clusters.' Below this are tabs for 'Compute instances', 'Compute clusters', 'Kubernetes clusters', and 'Attached'. The 'Compute instances' tab is selected. Below the tabs, it says 'Choose from a selection of CPU or GPU instances preconfigured with popular' followed by a link 'Learn more about compute instances'. Below this is a command bar with buttons: '+ New', 'Refresh', 'Start', 'Stop', and 'Restart'. Below the command bar is a search bar labeled 'Search'. Below the search bar is a table with columns 'Name', a star icon, and 'State'. The table contains one row with the name 'hexelo-aai-mls51153434', a green play button icon, and the state 'Running'.

Name	Star	State
hexelo-aai-mls51153434	🟢	Running

💡 Want to learn more? Review the documentation on [starting a compute instance](#).

- Obtain your **Subscription ID**, **Current Workspace**, and **Resource Group**, and then enter them in the following associated text boxes:

Subscription ID

Current Workspace

Resource Group



Expand this hint for guidance on obtaining your Subscription ID and Current Workspace.



Check your work

Verify

Create a compute instance for training

Hints Enabled

No Yes

- Using Administrator privileges, install Azure ML SDK.

```
pip install azure-ai-ml azure-identity
```

 Expand this hint for guidance on creating a Jupyter Notebook. 

- Open a terminal or command prompt and run as an Administrator.

```
pip install azure-ai-ml azure-identity
```

 Want to learn more? Review the documentation on .

 You can safely ignore the *WARNING: You are using pip version 21.2.3; however, version 25.1.1 is available. You should consider upgrading via the 'C:\Python310\python.exe -m pip install --upgrade pip' command* alert.

```
emetry-instrumentation-psycopy2-0.52b1 opentelemetry-instrumentation-requests-0.52b1 opentelemetry-instru
ib-0.52b1 opentelemetry-instrumentation-urllib3-0.52b1 opentelemetry-instrumentation-wsgi-0.52b1 openteler
detector-azure-0.1.5 opentelemetry-sdk-1.31.1 opentelemetry-semantic-conventions-0.52b1 opentelemetry-util
ackaging-25.0 psutil-6.1.1 pycparser-2.22 pydash-8.0.5 pyjwt-2.10.1 python-dateutil-2.9.0.post0 pyyaml-6.0
-0.36.2 requests-2.32.3 requests-oauthlib-2.0.0 rpds-py-0.24.0 six-1.17.0 strictyaml-1.7.3 tqdm-4.67.1 typ
-4.13.2 urllib3-2.4.0 wrapt-1.17.2 zipp-3.21.0
WARNING: You are using pip version 21.2.3; however, version 25.1.1 is available.
You should consider upgrading via the 'C:\Python310\python.exe -m pip install --upgrade pip' command.
C:\Windows\System32>_
```

 For the remainder of this Challenge Lab, use the Copy feature to to copy code-do *not* use the Type Text feature to enter your code, as it will introduce formatting errors to the inserted code.

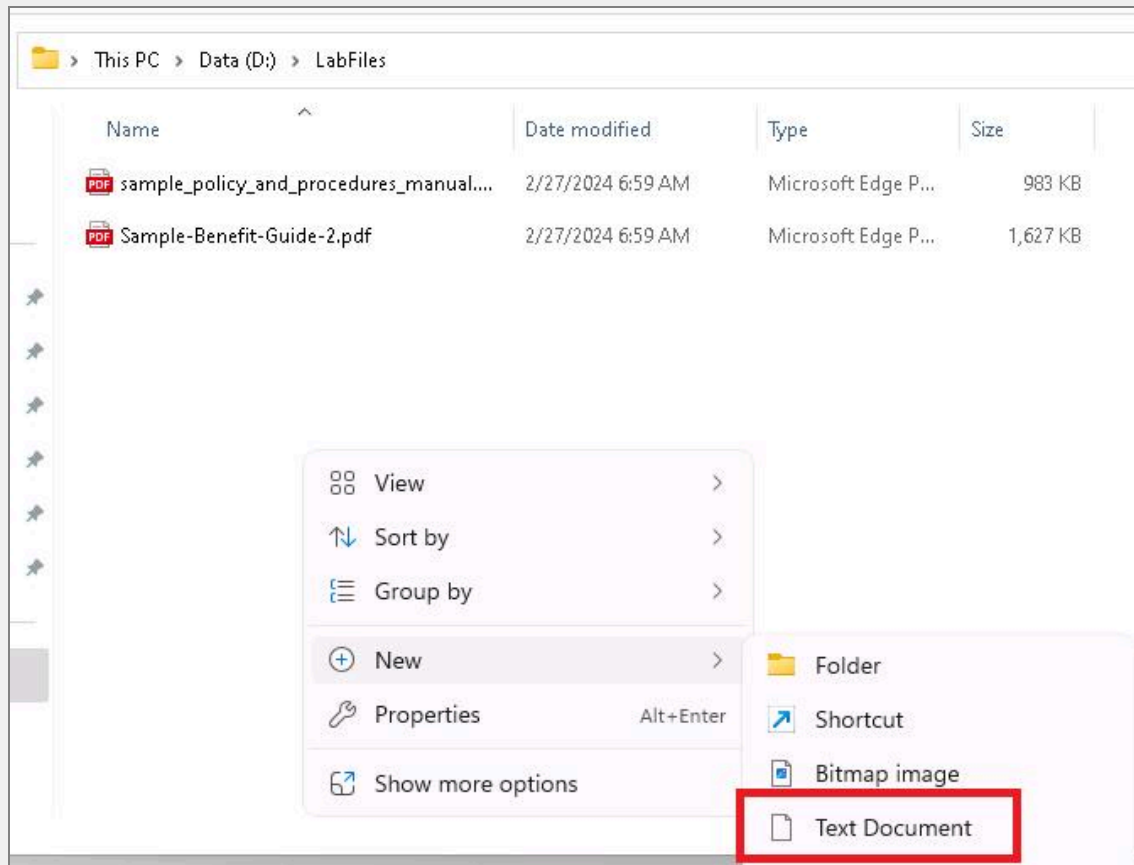
- Authenticate with Azure in Python by creating a new Python script named `connect.py` with the following script. Replace the three values with your actual **your-subscription-ID**, **your-resource-group**, and **workspace-name**, and then save in **D:\LabFiles**.

```
from azure.ai.ml import MLClient
from azure.identity import DefaultAzureCredential
```

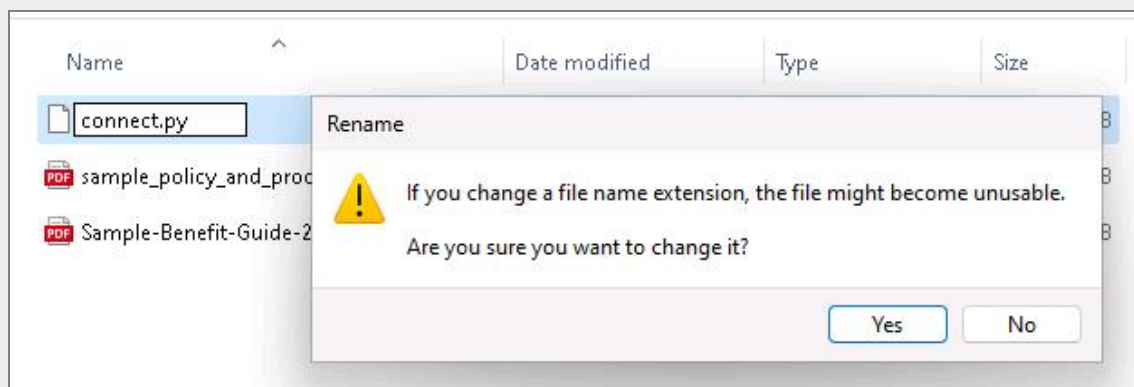
```
def get_ml_client():  
    credential = DefaultAzureCredential()  
    return MLClient(  
        credential=credential,  
        subscription_id="your-subscription-ID",  
        resource_group_name="your-resource-group",  
        workspace_name="workspace-name"  
    )
```

💡 Expand this hint for guidance on creating a handle to reference the workspace. ^

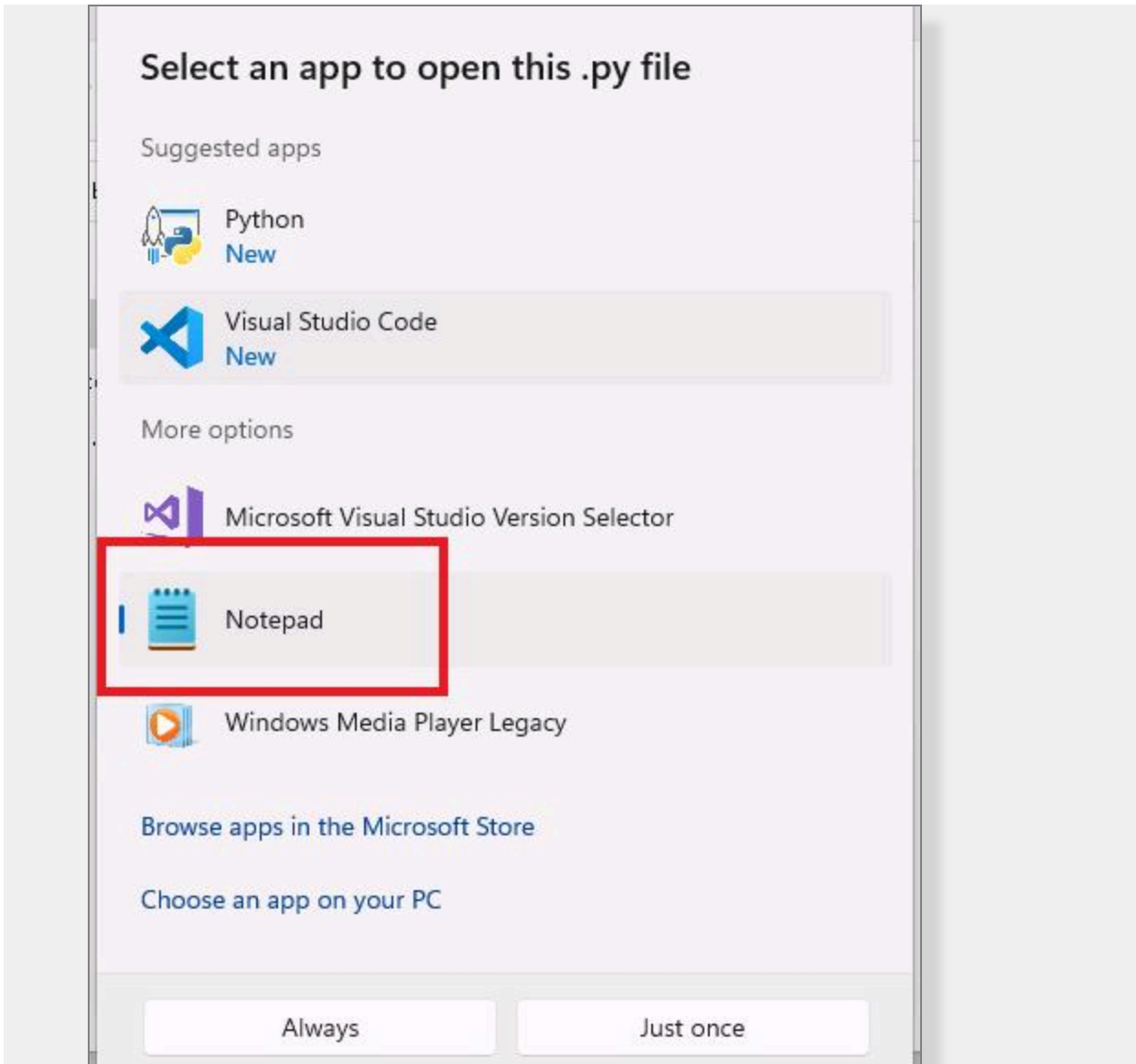
- On the Windows Desktop taskbar, select File Explorer, or select **Windows + E** to open a new Explorer window.
- Navigate to **T D:\LabFiles**, right-click, and then select **New | Text Document**.



- Rename the document **T connect.py**.
- On the *If you change a file name extension, the file might become unusable* alert, select **Yes**.



- Double-click **connect.py**, and then on the *Select an app to open this .py file* pop-up, select **Notepad**, and then select **Always**.



- Copy the above supplied script, and then paste it into the connect.py document.

```
connect.py
File Edit View

from azure.identity import DefaultAzureCredential
from azure.ai.ml import MLClient

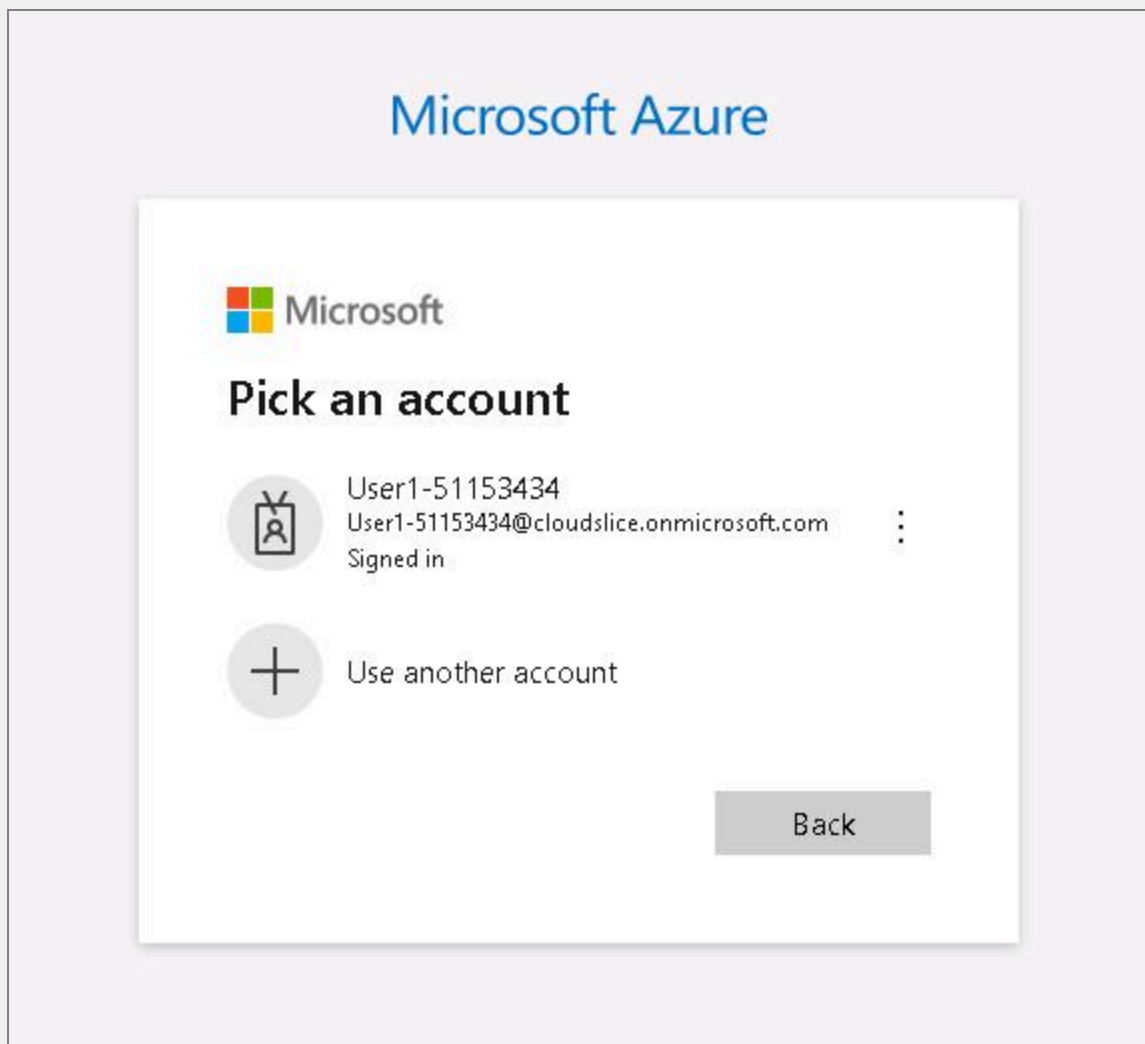
def get_ml_client():
    credential = DefaultAzureCredential()
    return MLClient(
        credential=credential,
        subscription_id="your-subscription-id", # TODO: Replace with your subscription ID
        resource_group_name="your-resource-group", # TODO: Replace with your resource group
        workspace_name="your-workspace-name" # TODO: Replace with your workspace name
    )
```

- Locate `subscription_id="your-subscription-id"`, and then replace **"your-subscription-id"** with your saved Subscription ID.
- Locate `resource_group_name="your-resource-group"`, and then replace **"your-resource-group"** with your saved Resource Group.
- Locate `workspace_name="your-workspace-name"`, and then replace **"your-workspace-name"** with your saved Workspace Name.
- Select **File | Save** to save `connect.py`.

- Perform Azure CLI Authentication by using the following script:

```
az login
```

-  This will launch a browser window for you to sign in. Once complete, `DefaultAzureCredential` will pick up your login from the CLI and work automatically.



- If necessary, verify Your Azure CLI Works by using the following script:

 `az account show`

 Creating MLClient will not connect to the workspace; it will wait for the first time it needs to make a call-this will happen in the next code cell.

```
C:\Windows\System32>az account show
{
  "environmentName": "AzureCloud",
  "homeTenantId": "8eb87a6e-8055-4135-b69d-f19c799ec045",
  "id": "40f9cd64-cf35-4b6f-ba20-7e6f12c6c528",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Skillable Original Content Dev 01",
  "state": "Enabled",
  "tenantId": "8eb87a6e-8055-4135-b69d-f19c799ec045",
  "user": {
    "name": "User1-51153434@cloudslice.onmicrosoft.com",
    "type": "user"
  }
}

C:\Windows\System32>
```

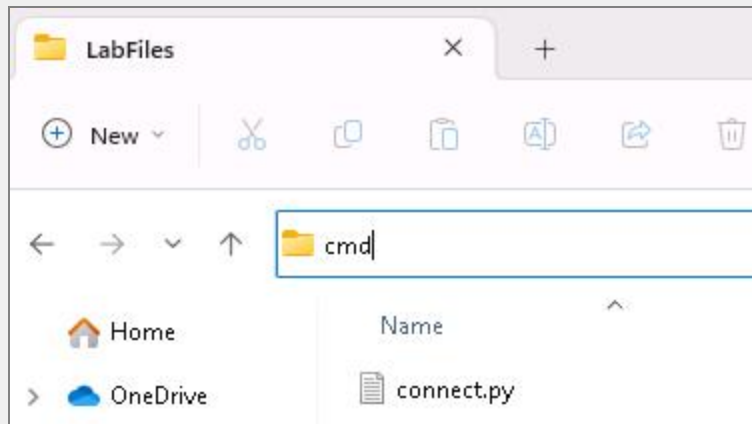
 Want to learn more? Review the documentation on [creating a handle to reference the workspace](#).

- Run the `connect.py` script once to confirm you can connect.

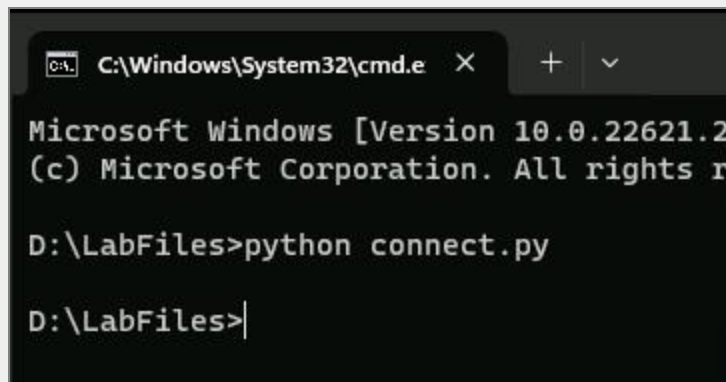
 `python connect.py`

💡 Expand this hint for guidance on running a python script.

- In the D:\LabFiles folder, in the address bar, select the entire address, enter `T cmd`, and then select **Enter**.



- In the Console, enter `T python connect.py`, and then select **Enter**.



- Create a Compute Instance (for training) named `T create_compute.py` by using the following script:

```
from azure.ai.ml.entities import AmlCompute
from connect import get_ml_client

ml_client = get_ml_client()

compute = AmlCompute(
    name="ml-compute-self", # You can change this name if needed
    size="Standard_DS11_v2",
    min_instances=0,
    max_instances=1
)

ml_client.begin_create_or_update(compute).wait()
print("Compute resource created or updated successfully.")
```

💡 Expand this hint for guidance on creating a compute instance. ^

- In the `T D:\LabFiles` folder, right-click, and then select **New | Text Document**.
- Rename the document `T create_compute.py`.
- On the *If you change a file name extension, the file might become unusable* alert, select **Yes**.
- Double-click **create_compute.py**, copy the above supplied script, and then paste it into the `create_compute.py` document.
- Select **File | Save** to save `create_compute.py`.

💡 Want to learn more? Review the documentation on [adding a code cell to a Jupyter notebook](#).

- Run the `T create_compute.py` script.

📄 `python create_compute.py`

📄 This can take a few minutes. Once done, your compute is ready for training jobs.

```
D:\LabFiles>python create_compute.py
Compute resource created or updated successfully.

D:\LabFiles>|
```

Check your work

Verify

Set up a machine learning job to train and register a model

Hints Enabled

No Yes

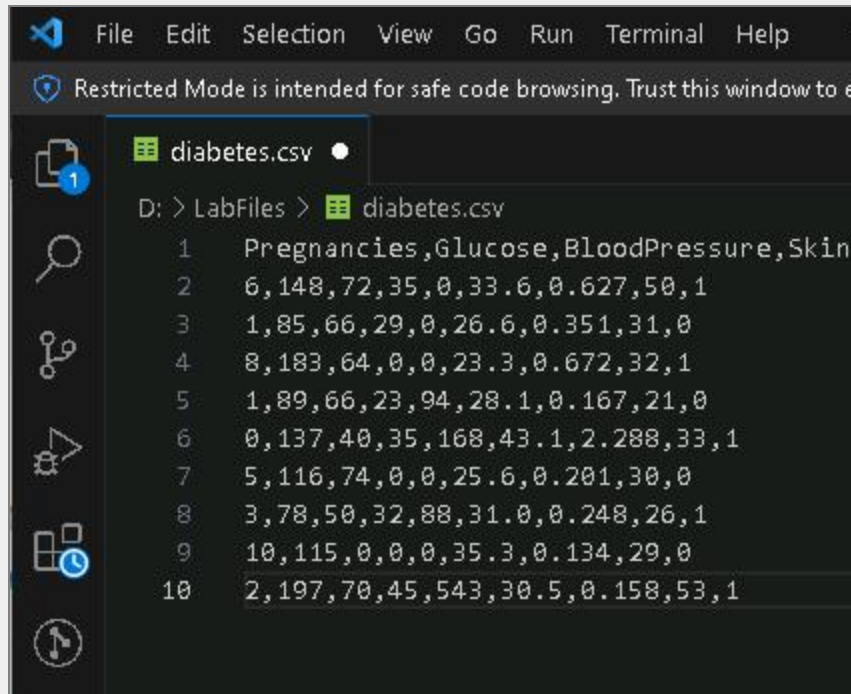
- Create a sample `diabetes.csv` file you can use for testing the upload process by using the following text:



```
Pregnancies,Glucose,BloodPressure,SkinThickness,Insulin,BMI,DiabetesPedigree
6,148,72,35,0,33.6,0.627,50,1
1,85,66,29,0,26.6,0.351,31,0
8,183,64,0,0,23.3,0.672,32,1
1,89,66,23,94,28.1,0.167,21,0
0,137,40,35,168,43.1,2.288,33,1
5,116,74,0,0,25.6,0.201,30,0
3,78,50,32,88,31.0,0.248,26,1
10,115,0,0,0,35.3,0.134,29,0
2,197,70,45,543,30.5,0.158,53,1
```

💡 Expand this hint for guidance on creating a compute instance.

- In the `D:\LabFiles` folder, right-click, and then select **New | Text Document**.
- Rename the document `diabetes.csv`.
- On the *If you change a file name extension, the file might become unusable* alert, select **Yes**.
- Double-click **diabetes.csv** to open a code editor (VS Code, Notepad), copy the above supplied script, and then paste it into the diabetes.csv document.

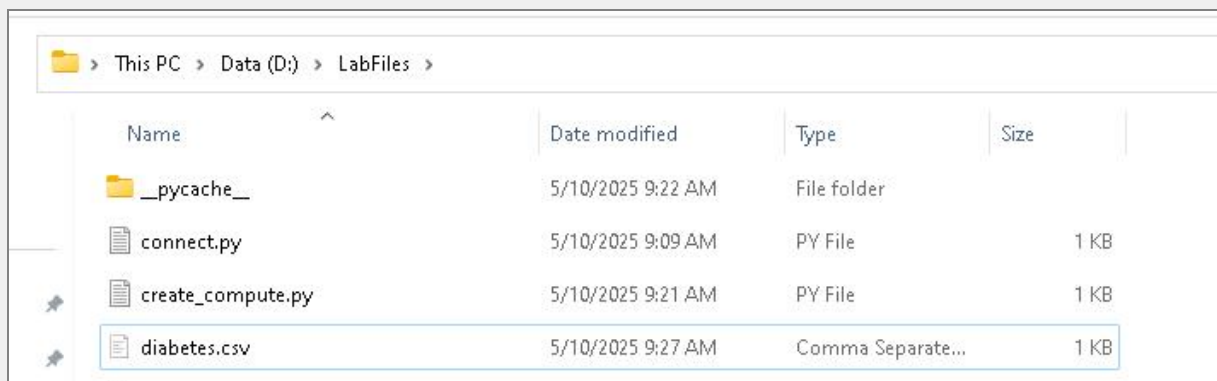


The screenshot shows a VS Code editor window with the file `diabetes.csv` open. The file path is `D:\LabFiles > diabetes.csv`. The content of the file is a CSV dataset with 10 rows of data. The first row is the header: `Pregnancies,Glucose,BloodPressure,SkinT`. The subsequent rows contain numerical values for each feature.

```
1 Pregnancies,Glucose,BloodPressure,SkinT
2 6,148,72,35,0,33.6,0.627,50,1
3 1,85,66,29,0,26.6,0.351,31,0
4 8,183,64,0,0,23.3,0.672,32,1
5 1,89,66,23,94,28.1,0.167,21,0
6 0,137,40,35,168,43.1,2.288,33,1
7 5,116,74,0,0,25.6,0.201,30,0
8 3,78,50,32,88,31.0,0.248,26,1
9 10,115,0,0,0,35.3,0.134,29,0
10 2,197,70,45,543,30.5,0.158,53,1
```

- Select **Ctrl + S** to save diabetes.csv, and then close the text editor.

📁 Make sure your dataset diabetes.csv is saved in the same directory.



The screenshot shows a Windows File Explorer window with the address bar set to `This PC > Data (D:) > LabFiles >`. The table below lists the files and folders in this directory.

Name	Date modified	Type	Size
__pycache__	5/10/2025 9:22 AM	File folder	
connect.py	5/10/2025 9:09 AM	PY File	1 KB
create_compute.py	5/10/2025 9:21 AM	PY File	1 KB
diabetes.csv	5/10/2025 9:27 AM	Comma Separate...	1 KB

- Create a file named `upload_data.py`, and then upload Your Dataset by using the following script:

```
from azure.ai.ml.entities import Data
from azure.ai.ml.constants import AssetTypes
from connect import get_ml_client

ml_client = get_ml_client()

# Register the dataset
dataset = Data(
    path="diabetes.csv", # Make sure this is the correct relative path
    type=AssetTypes.URI_FILE,
    description="Diabetes dataset for training",
    name="diabetes-data"
)

ml_client.data.create_or_update(dataset)
print("✅ Data uploaded and registered successfully.")
```

💡 Expand this hint for guidance on uploading a Dataset.

- In the `D:\LabFiles` folder, right-click, and then select **New | Text Document**.
- Rename the document `upload_data.py`.
- On the *If you change a file name extension, the file might become unusable* alert, select **Yes**.
- Double-click **upload_data.py**, copy the above supplied script, and then paste it into the `upload_data.py` document.
- Select **File | Save** to save `upload_data.py`.

- Run the `upload_data.py` script.

```
python upload_data.py
```

```
D:\LabFiles>python upload_data.py
Uploading diabetes.csv (< 1 MB): 100%|#####| 375/375 [00:00<00:00, 20.7kB/s]

Data uploaded and registered.
D:\LabFiles>
```

- Create a new folder in **D:\LabFiles** named `src`, and then create a Training Script named `train.py` by using the following script:



```
import pandas as pd
import argparse
from sklearn.linear_model import LogisticRegression
import joblib

parser = argparse.ArgumentParser()
parser.add_argument("--input_data", type=str, help="Path to input CSV")
args = parser.parse_args()

df = pd.read_csv(args.input_data)
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

model = LogisticRegression(max_iter=500)
model.fit(X, y)

joblib.dump(model, "model.joblib")
```



Expand this hint for guidance on creating a Training script.



- In the **T** D:\LabFiles folder, right-click, and then select **New | Folder**.
- Rename the New Folder **T** src, and then select **Enter**.
- Double-click the **src** folder.
- In the **T** D:\LabFiles\src folder, right-click, and then select **New | Text Document**.
- Rename the document **T** train.py.
- On the *If you change a file name extension, the file might become unusable* alert, select **Yes**.
- Double-click **upload_data.py**, copy the above supplied script, and then paste it into the train.py document.
- Select **File | Save** to save train.py.



This script trains a model and saves it as model.joblib.

- Switch to the Administrator: Command Prompt, and run the following **T** pip install pandas scikit-learn joblib script:



```
pip install pandas scikit-learn joblib
```

- Switch back to the D:\LabFiles Command prompt, and then run the **T** train.py python script.



```
python train.py --input_data ../diabetes.csv
```

- Create a new Python script named `submit_job.py` in the **D:\LabFiles** folder by using the following script. Replace the `subscription_id`, `resource_group_name`, and `workspace_name` with your actual **your-subscription-ID**, **your-resource-group**, and **workspace-name**, and then save.

```

from azure.ai.ml import MLClient, command, Input
from azure.identity import DefaultAzureCredential

# Connect to Azure ML
from connect import get_ml_client # This must match exactly
ml_client = get_ml_client() # This calls the function you defined

# Create the job with input data path
job = command(
    code="./src",
    command="python train.py --input_data ${inputs.diabetes_data}",
    inputs={
        "diabetes_data": Input(
            type="uri_file",
            path="diabetes.csv" # or the registered data asset name
        )
    },
    environment="AzureML-sklearn-1.0-ubuntu20.04-py38-cpu@latest",
    compute="ml-compute-self",
    display_name="diabetes-training",
    experiment_name="diabetes-exp"
)

# Submit
returned_job = ml_client.jobs.create_or_update(job)
ml_client.jobs.stream(returned_job.name)

```

💡 Expand this hint for guidance on uploading a Dataset. ^

- Return to the `D:\LabFiles` folder, right-click, and then select **New | Text Document**.
- Rename the document `submit_job.py`.
- On the *If you change a file name extension, the file might become unusable* alert, select **Yes**.
- Double-click **submit_job.py**, copy the above supplied script, and then paste it into the `upload_data.py` document.
- Locate `subscription_id="your-subscription-id"`, and then replace **"your-subscription-id"** with your saved Subscription ID.
- Locate `resource_group_name="your-resource-group"`, and then replace **"your-resource-group"** with your saved Resource Group.
- Locate `workspace_name="your-workspace-name"`, and then replace **"your-workspace-name"** with your saved Workspace Name.
- Select **File | Save** to save `submit_job.py`.

- Run the `submit_job.py` script.

📄 `python submit_job.py`

```
Uploading src (0.0 MBs): 100%|#####| 513/513 [00:00<00:00, 32801.47it/s]

RunId: plucky_night_gb51cg6rnr
Web View: https://ml.azure.com/runs/plucky_night_gb51cg6rnr?wsid=/subscriptions/40f9cd64-cf35-4b6f-ba20-7e6f12c6c528/resourcegroups/RG1lod51153434/workspaces/HexeloAML-workspace51153434

Execution Summary
=====
RunId: plucky_night_gb51cg6rnr
Web View: https://ml.azure.com/runs/plucky_night_gb51cg6rnr?wsid=/subscriptions/40f9cd64-cf35-4b6f-ba20-7e6f12c6c528/resourcegroups/RG1lod51153434/workspaces/HexeloAML-workspace51153434

D:\LabFiles>
```

📄 Wait for the training job to complete.

Check your work

Verify

Deploy the model to a REST Endpoint

Hints Enabled

No Yes

- Create `register_model.py` and register the model **model.joblib** Trained Model, once the training job is complete, by running the following:

```
from connect import get_ml_client # This must match exactly

ml_client = get_ml_client() # This calls the function you defined

from azure.ai.ml.entities import Model

model = Model(
    path="src/model.joblib", # Change if path is different
    name="diabetes-model",
    description="Logistic regression model for diabetes prediction"
)

ml_client.models.create_or_update(model)
```

- Run the **register_model.py** script.

```
python register_model.py
```

- Create a `src/score.py` file named `score.py`.

```
import joblib
import numpy as np
import pandas as pd
import os

def init():
    global model
    model_path = os.path.join(os.getenv("AZUREML_MODEL_DIR"), "model.joblib")
    model = joblib.load(model_path)

def run(raw_data):
    try:
        data = pd.DataFrame(raw_data["data"], columns=raw_data["columns"])
        predictions = model.predict(data)
        return predictions.tolist()
```

```
except Exception as e:
    return str(e)
```

- Create environment:

```
name: diabetes-env
dependencies:
  - python=3.8
  - pip
  - numpy
  - pandas
  - scikit-learn
  - pip:
    - joblib
    - azureml-inference-server-http
```

- `az ml online-endpoint delete --name diabetes-endpoint-2a47b199 --workspace-name HexeloAML-workspace51194407 --resource-group RG1lod51194407 --yes`

- Create `deploy_model.py` and deploy to a REST Endpoint by running the following:

```
from azure.ai.ml import MLClient
from azure.ai.ml.entities import ManagedOnlineEndpoint, ManagedOnlineDeployment
from azure.identity import DefaultAzureCredential
import uuid

# Step 1: Connect to Azure ML
from connect import get_ml_client # This must match exactly

ml_client = get_ml_client() # This calls the function you defined

# Step 2: Generate a unique endpoint name
endpoint_name = f"diabetes-endpoint-{uuid.uuid4().hex[:8]}"
print(f"Creating endpoint: {endpoint_name}")

# Step 3: Define the endpoint
endpoint = ManagedOnlineEndpoint(
    name=endpoint_name,
    auth_mode="key"
)
ml_client.begin_create_or_update(endpoint).wait()

# Step 4: Load the registered model
model = ml_client.models.get(name="diabetes-model", label="latest")

# Step 5: Define an inference environment
env = ml_client.environments.get(name="AzureML-sklearn-1.0-ubuntu20.04-py38")

# Step 6: Create the deployment
```



```

deployment = ManagedOnlineDeployment(
    name="blue",
    endpoint_name=endpoint_name,
    model=model,
    environment=env,
    code_path="src", # contains score.py
    scoring_script="score.py",
    instance_type="Standard_DS3_v2",
    instance_count=1
)

ml_client.begin_create_or_update(deployment).wait()

print(f" Deployment successful: {endpoint_name}")

```

- Run the **deploy_model.py** script.

```
python deploy_model.py
```

- Create a JSON file named `sample-request.json` and test the Endpoint by running the following:

```

{
  "input_data": {
    "columns": ["Pregnancies", "Glucose", "BloodPressure", "SkinThickness",
    "data": [[6, 148, 72, 35, 0, 33.6, 0.627, 50]]
  }
}

```

- Locate your Primary key and scoring URL:

```
az ml online-endpoint get-credentials --name diabetes-endpoint-580cf9fa --w
```

- Create `test_endpoint.py`

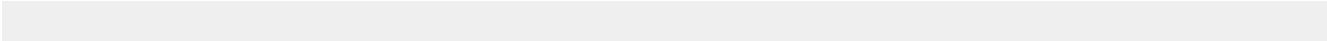
```

response = ml_client.online_endpoints.invoke(
    endpoint_name="your-endpoint-name",
    request_file="sample-request.json"
)
print(response)

```

Replace "your-endpoint-name" with the printed name from the deploy step.

Check your work


Verify

Clean up your resources

- Clean up your resources by adding and running the following code:

 `ml_client.online_endpoints.begin_delete(name=online_endpoint_name)`

Summary

Congratulations, you have completed the **Train and Deploy a Machine Learning Model in Azure ML** Challenge Lab.

You have accomplished the following:

- Created a compute instance.
- Set up a machine learning job to train and register a model.
- Deployed a model as an online endpoint.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

 Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to [Challenge Labs feedback](#).

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- [Manage Azure Resource Deployment by Using an Azure Resource Manager Template \[Guided\]](#)
- [Manage Azure Resource Groups \[Guided\]](#)
- [Create an Azure Function App \[Guided\]](#)
- [Create an Azure Logic App \[Guided\]](#)
- [Configure an Azure Distributed Denial of Service Protection Plan \[Guided\]](#)
- [Manage Encryption by Using an Azure Key Vault \[Guided\]](#)
- [Configure Azure Role-Based Access Control \[Guided\]](#)

- Configure an Azure Lock [Guided]
- Configure Monitoring by Using Azure Monitor [Guided]
- Deploy an Azure Virtual Machine [Guided]
- View Azure Service Health Options [Guided]
- Manage Microsoft Entra Users and Groups [Guided]
- Implement a Network Security Group [Guided]
- Azure Cost Management [Guided]

